

Theorem [theorem] Proposition [theorem] Corollary [theorem] Lemma [theorem] Definition

Proof-Bound Enterprise Change: Making Invalid State Transitions Mathematically Inaccessible

Sean Chatman

wasm4pm Project
info@chatmangpt.com

Abstract. Enterprise systems currently permit change through any surface that can influence state: form fields, API calls, direct database writes, workflow overrides, and increasingly, AI-agent tool calls. Auditing reconstructs what happened after the fact. We invert this discipline. Extending van der Aalst’s process mining model of admitted symbolic events and boundary predicates into the domain of enterprise change governance, we define the *admitted change universe* $A_{\text{enterprise}}$ — the set of signed, typed, object-linked, process-admitted, policy-bound, receipt-valid changes — and prove the main theorem: $\text{Accept}(x) \Rightarrow x \in A_{\text{enterprise}}$. Under this law, eight classes of adversarial mutation — including raw database writes, process-state skips, object substitution, backdating, replay, schema-valid fraud, rogue admin edits, and AI-agent hallucinated actions — become inaccessible to acceptance without breaking a cryptographic, algorithmic, or implementation assumption we name explicitly. We implement the framework in *wasm4pm*, a deterministic WebAssembly process mining platform, using BLAKE3 receipt chains, OCEL 2.0 object-centric event logs, and a boundary predicate instantiated as thirteen non-overlapping refusal states. The contribution is not a claim of invincibility; it is a formal reduction of the adversary’s goal to a disjunction of named, bounded assumptions.

Keywords: process mining · enterprise change governance · receipt chains · admissibility · post-quantum cryptography · OCEL 2.0 · boundary predicates

1 Introduction

Enterprise systems face a structural problem: the surfaces through which change enters the system are too many, too permeable, and too decoupled from the processes that are supposed to govern them. An attacker who gains database access can issue `UPDATE invoice SET status = 'paid'` without any approval event having occurred. A privileged administrator can promote a user to admin directly, bypassing the workflow that was designed to require a second approval. An AI agent, given a tool that calls a write API, can modify enterprise state as a side-effect of any prompt it is given — with no cryptographic binding to its declared mandate.

Current defences respond after the fact: logs are written, anomaly detectors flag deviations, and audit trails attempt to reconstruct what happened. Process mining, in van der Aalst’s canonical formulation, discovers and verifies process models against event logs recorded after execution [2]. The insight of this paper is to *invert the temporal position* of that discipline: instead of mining a log to check whether a completed process conformed, we require that a process-valid event be produced and admitted *before* the change is allowed to alter enterprise state.

Contributions

This paper makes four contributions:

- (1) **The Enterprise Change Admissibility Framework.** We define the admitted change universe $A_{\text{enterprise}}$ as a Cartesian product of actor, event type, object relation, process state, prior receipt, policy version, validator version, nonce, output receipt, and signature. We state and prove the main theorem: $\text{Accept}(x) \Rightarrow x \in A_{\text{enterprise}}$.
- (2) **Eight inaccessibility propositions.** We prove that eight classes of adversarial mutation — raw database writes, process-state skips, object substitution, backdating, replay, schema-valid fraud, rogue admin edits, and AI-agent hallucinated actions — are inaccessible to acceptance under the framework. Each proposition identifies the specific predicate component that fails.
- (3) **A named attack decomposition.** We bound the probability of accepting an invalid change as a disjunction of seven named assumptions, each with a distinct security category. This replaces vague security claims with a falsifiable structure.
- (4) **A reference implementation.** We instantiate the framework in *wasm4pm* [9], a deterministic WebAssembly process mining platform, using BLAKE3 receipt chains, OCEL 2.0 object-centric event logs [7], and a boundary predicate expressed as thirteen non-overlapping refusal states.

Structure

Section 2 covers process mining, OCEL 2.0, receipt chains, and post-quantum cryptographic signing. Section 3 develops the formal model and proves the main

theorem. Section 4 proves the eight inaccessibility propositions. Section 5 gives the attack decomposition. Section 6 describes the wasm4pm implementation. Section 7 surveys related work. Section 8 concludes.

2 Background

2.1 Process Mining and OCEL 2.0

Process mining operates on event logs to discover process models, check conformance, and enhance existing models [2]. The canonical event log L is a multiset of traces, each trace being a sequence of events over a set of activities. An event e carries at minimum an activity label $a(e)$, a timestamp $t(e)$, and a case identifier $c(e)$.

Object-Centric Event Logs (OCEL 2.0) [11,7] generalise this model: an event e is associated with a *set* of typed objects $O(e) \subseteq \mathcal{O}$, removing the single-case assumption that forces artificial case notions on multi-entity processes. Formally, an OCEL is a tuple $(\mathcal{E}, \mathcal{O}, \mathcal{A}, \mathcal{T}, evAttr, objAttr)$ where $\mathcal{E}$ is a set of events, $\mathcal{O}$ a set of objects, $\mathcal{A}$ a set of activities, and $\mathcal{T}$ a set of object types [7].

The *boundary predicate* $B : S \times E \rightarrow \{0, 1\}$, where S is the set of process states and E the set of event types, encodes whether an event is admissible from a given state. If $B(s, e) = 0$, the state transition function is required to satisfy $\mu(s, e) = s$ — the event produces no change. Boundary predicates appear implicitly throughout conformance checking literature (e.g., as guard conditions on Petri net transitions [2]); we make them an explicit first-class component of the admissibility model.

2.2 Receipt Chains

A *receipt* R_t is a cryptographic record that the state transition $s_t \xrightarrow{e_t} s_{t+1}$ occurred with specific inputs and outputs. We write $R_t \vdash s_{t+1} = \mu(s_t, e_t)$ to mean that receipt R_t witnesses the transition. Receipts compose into a chain:

$$R_0 \rightarrow R_1 \rightarrow \cdots \rightarrow R_n$$

where R_t includes the hash $h(R_{t-1})$ of its predecessor, the input hash $h(in_t)$, the output hash $h(out_t)$, and the run identifier rid_t . The chain is BLAKE3-hashed throughout; BLAKE3 offers 256-bit security with verified resistance to length-extension attacks and is suitable for streaming computation over large event logs.

A receipt is *not* a receipt if it carries only hash values without the backing event data. The *axiom of receipt substance* states:

A receipt containing only expected hash, observed hash, alignment state, and receipt hash is an unverifiable claim, not evidence.

Accordingly, a valid receipt in our framework must embed or content-addressably reference the full OCEL 2.0 logs it witnesses. We formalise the consequence in Definition 2 (Section 3).

The proof class hierarchy orders receipts by evidential strength: `FIXTUREPROOF` $\prec$ `PREDICATEPROOF` $\prec$ `CHICAGOPROOF` $\prec$ `BOUNDARYPROOF`. A receipt may not claim a higher class unless all conditions of that class are satisfied (refusal state `ProofClassOverclaimed`).

2.3 Post-Quantum Cryptographic Signing

Classic ECDSA and RSA signatures are vulnerable to Shor’s algorithm on a sufficiently large quantum computer [5]. Enterprise receipt chains must be verifiable for years or decades; a receipt signed today may need to be re-verified after quantum computing becomes practical. NIST’s 2024 post-quantum standards address this: FIPS 204 specifies *ML-DSA* (Module-Lattice-Based Digital Signature Algorithm, based on CRYSTALS-Dilithium) [15] and FIPS 205 specifies *SLH-DSA* (Stateless Hash-Based Digital Signature Algorithm, based on SPHINCS+) [16].

We model the signing operation abstractly as $\sigma = \text{Sign}(sk, h(e\|O\|P\|R\|V))$ where sk is the signer’s secret key, e the event, O the object set, P the policy version, R the prior receipt, and V the validator version. Verification is $\text{VerifySig}(\sigma, pk, m) \in \{0, 1\}$. The concrete algorithm is ML-DSA-44 or higher; the abstraction is algorithm-agnostic.

3 Formal Model

We now develop the formal model. Let $\mathcal{A}$ be a set of actors, $\mathcal{E}$ a set of event types, $\mathcal{O}$ a set of typed object sets, $\mathcal{S}$ a set of process states, $\mathcal{P}$ a set of policy versions, $\mathcal{V}$ a set of validator versions, and Σ a set of signatures.

Definition 1 (Enterprise Change Universe). *The enterprise change universe is:*

$$\Omega_{\text{change}} = \mathcal{A} \times \mathcal{E} \times \mathcal{O} \times \mathcal{S} \times \mathcal{R} \times \mathcal{P} \times \mathcal{V} \times \mathcal{T} \times \mathcal{N} \times \mathcal{R} \times \Sigma$$

where $\mathcal{R}$ is the set of receipts, $\mathcal{T}$ the set of timestamps, and $\mathcal{N}$ the set of nonces. A candidate change is a tuple $x = (a, e, O, s, R_{\text{prior}}, p, v, t, n, R, \sigma)$.

Definition 2 (Valid Receipt). *A receipt R is valid for transition $s \xrightarrow{e} s'$ if: (i) R embeds or content-addressably references the full OCEL 2.0 logs for the transition; (ii) R carries the hashes $h(R_{\text{prior}})$, $h(in)$, $h(out)$; (iii) the alignment state is computed by the verifier, not self-certified by the producer; and (iv) a challenge nonce minted before execution appears bound into the observed event payload. We write $\text{VerifyReceipt}(R) = 1$.*

Definition 3 (Acceptance Predicate). *The acceptance predicate for a candidate change $x = (a, e, O, s, R_{\text{prior}}, p, v, t, n, R, \sigma)$ is:*

$$\begin{aligned} \text{Accept}(x) = & \text{VerifySig}(\sigma, h(a\|e\|O\|s\|R_{\text{prior}}\|p\|v\|t\|n)) \\ & \wedge \text{VerifyReceipt}(R) \\ & \wedge \text{PolicyAllowed}(p) \\ & \wedge \text{ValidatorAllowed}(v) \\ & \wedge \text{Fresh}(n) \\ & \wedge B(s, e) = 1 \\ & \wedge \text{ObjectsValid}(O) \end{aligned}$$

where $\text{Fresh}(n)$ holds iff nonce n has not been consumed in any prior accepted change, and $\text{ObjectsValid}(O)$ holds iff the object-event relation encoded in O is consistent with the OCEL 2.0 schema and signed into σ .

Definition 4 (Admitted Change Set). *The admitted change set is:*

$$A_{\text{enterprise}} = \{x \in \Omega_{\text{change}} \mid \text{Accept}(x) = 1\}$$

Theorem 1 (Acceptance Implies Membership). *For any candidate change $x \in \Omega_{\text{change}}$:*

$$\text{Accept}(x) = 1 \Rightarrow x \in A_{\text{enterprise}}$$

Proof. By Definition 4, $A_{\text{enterprise}}$ is exactly the set of x for which $\text{Accept}(x) = 1$. The implication follows by set membership. $\square$

While Theorem 1 is immediate by definition, its force is *operational*: it states that the acceptance gate is the sole path from Ω_{change} to enterprise state. No change enters without passing every conjunct of Accept .

Corollary 1 (Refusal or Residualization of Non-Members). *For any candidate change $x \in \Omega_{\text{change}}$:*

$$x \notin A_{\text{enterprise}} \Rightarrow \text{Refuse}(x) \vee \text{Residualize}(x)$$

Here $\text{Refuse}(x)$ means the change is rejected outright and $\text{Residualize}(x)$ means it is quarantined as a *residual* — recorded as an out-of-band observation without being incorporated into enterprise state. Residualization is the lossless option for changes that carry partial evidence: the data is preserved for investigation but is not state-authoritative.

Definition 5 (State Transition Law). *The enterprise state transition function $\mu : S \times E \rightarrow S$ satisfies:*

$$\mu(s, e) = \begin{cases} s' & \text{if } B(s, e) = 1 \text{ and } \text{Accept}(x) = 1 \text{ for the change carrying } e \\ s & \text{otherwise} \end{cases}$$

Transitions that do not satisfy Accept leave state unchanged.

The Chesterton Fence as a State Law

Chesterton’s Fence principle [10] holds that a boundary should not be removed until its purpose is understood. The boundary predicate $B(s, e)$ is the formal expression of this principle for enterprise state: every transition is fenced, and the fence may only be crossed by a change that satisfies all of Accept. Crucially, the fence does not prevent the *attempt* — an adversary may submit any $x \in \Omega_{\text{change}}$ — but it prevents the attempt from *counting* as valid state unless the witness is produced.

4 Inaccessibility of Invalid Change Classes

We prove that eight classes of adversarial mutation are inaccessible to acceptance under the framework. For each class we give: the attack description, the component of Accept that fails, and the formal statement.

Proposition 1 (Unreceipted Raw Mutation). *A direct state mutation (e.g., a raw SQL UPDATE) that does not produce a valid receipt is inaccessible to acceptance.*

Proof. Such a mutation is submitted as x with $\text{VerifyReceipt}(R) = 0$: no OCEL 2.0 log is embedded or referenced, and no challenge nonce was minted before execution. By Definition 3, $\text{Accept}(x) = 0$. By Corollary 1, the change is refused or residualized. $\square$

Proposition 2 (Process-State Skip). *A change that attempts a state transition not admitted from the current process state (e.g., $\text{draft} \rightarrow \text{paid}$ when only $\text{draft} \rightarrow \text{submitted}$ is lawful) is inaccessible to acceptance.*

Proof. Let $s_{\text{current}} = \text{draft}$ and $e = \text{MarkPaid}$. The boundary predicate satisfies $B(\text{draft}, \text{MarkPaid}) = 0$. By Definition 3, $\text{Accept}(x) = 0$. By Definition 5, $\mu(s, e) = s$; state does not change. $\square$

Proposition 3 (Object Substitution). *A change that binds an event to the wrong object (e.g., an approval attached to a different invoice from the one the approver saw) is inaccessible to acceptance.*

Proof. The signature σ is computed over $h(a\|e\|O\|\dots)$ where O encodes the canonical OCEL 2.0 object-event relation. Substituting a different object set O' invalidates σ : $\text{VerifySig}(\sigma, h(\dots\|O'\|\dots)) = 0$. Alternatively, if the attacker re-signs with O' , they must forge the authority key, which reduces to breaking ML-DSA. Hence $\text{Accept}(x) = 0$. $\square$

Proposition 4 (Backdated Change). *A change claiming to have occurred at an earlier time is inaccessible to acceptance as valid state.*

Proof. The receipt chain imposes causal order: R_t includes $h(R_{t-1})$. A backdated change would need to be inserted at position $k < t$ in the chain, requiring a collision in $h(R_{k-1})$ (BLAKE3 preimage resistance) or re-signing all downstream receipts $R_k, \dots, R_t$ with valid authority keys. The timestamp field t is also bound into σ , so re-dating without re-signing fails `VerifySig`. $\square$

Proposition 5 (Replay). *A previously accepted valid change cannot be replayed as a new acceptance.*

Proof. A replayed event carries the original nonce n , which has already been consumed. $\text{Fresh}(n) = 0$, so $\text{Accept}(x) = 0$. Furthermore, σ binds R_{prior} : if the process has advanced since the original acceptance, the prior receipt in σ no longer matches the current chain head, causing `VerifySig` to fail. $\square$

Proposition 6 (Schema-Valid Fraud). *A change that is syntactically and schema-valid but process-invalid is inaccessible to acceptance.*

Proof. Schema validity is a necessary but not sufficient condition for `Accept`. The conjunct $B(s, e) = 1$ requires the event to be admissible from the current process state, regardless of whether its payload is well-formed. A schema-valid `{"status": "approved"}` is refused whenever $B(s_{\text{current}}, \text{ApprovalGranted}) = 0$. $\square$

Proposition 7 (Rogue Admin Mutation). *An administrator cannot directly alter enterprise state without satisfying the admitted-change witness.*

Proof. Administrative privilege in the framework means the authority to emit events of privileged types; it does not mean the authority to bypass `Accept`. The change x produced by an admin must still satisfy `VerifySig`, `VerifyReceipt`, $B(s, e) = 1$, `PolicyAllowed`, and `ValidatorAllowed`. Administrative authority is encoded in the actor component a and checked by `VerifySig` — the admin is inside the process, not above it. $\square$

Proposition 8 (AI-Agent Hallucinated Action). *An action produced by an AI agent that does not carry a valid admitted-change witness is inaccessible to acceptance as enterprise state.*

Proof. An agent action is a candidate change x_{agent} . If the agent cannot produce a valid σ , a fresh nonce n , a receipt R with embedded OCEL logs, and satisfy $B(s, e) = 1$, then $\text{Accept}(x_{\text{agent}}) = 0$ and the action is residualized. The agent may continue operating, but its output does not enter enterprise state. This converts agents from open-ended mutation engines into receipt-bearing process participants. $\square$

Table 1 summarises the failing predicate for each class.

Table 1. Attack classes and the predicate component that blocks acceptance.

Attack class	Failing predicate component
Raw database/API write	$\text{VerifyReceipt}(R) = 0$
Process-state skip	$B(s, e) = 0$
Object substitution	$\text{VerifySig}(\sigma, \cdot) = 0$ (forged relation)
Backdating	$h(R_{k-1})$ collision or VerifySig on t
Replay	$\text{Fresh}(n) = 0$
Schema-valid fraud	$B(s, e) = 0$
Rogue admin direct edit	VerifySig or PolicyAllowed
AI-agent hallucinated action	VerifyReceipt or $B(s, e) = 0$

5 Attack Surface Decomposition

The inaccessibility propositions of Section 4 are conditional on the underlying cryptographic and implementation assumptions holding. We now make those assumptions explicit and bound the probability of accepting an invalid change as a disjunction over named failure channels.

Theorem 2 (Attack Surface Decomposition). *The probability that an adversary causes an invalid change to be accepted satisfies:*

$$\begin{aligned}
\Pr[\exists x \notin A_{\text{enterprise}} : \text{Accept}(x) = 1] &\leq \Pr[\text{PQC forge}] \\
&\quad + \Pr[\text{receipt forge}] \\
&\quad + \Pr[\text{boundary bypass}] \\
&\quad + \Pr[\text{policy gap}] \\
&\quad + \Pr[\text{validator compromise}] \\
&\quad + \Pr[\text{object-model gap}] \\
&\quad + \Pr[\text{implementation bypass}]
\end{aligned}$$

Proof. By a union bound over all conjuncts of Accept : the adversary must cause at least one conjunct to evaluate incorrectly. Each term corresponds to exactly one failure mode enumerated below. $\square$

We characterise each channel.

PQC forge. The adversary forges a valid ML-DSA or SLH-DSA signature without possessing the authority secret key. Under the hardness of Module-LWE (ML-DSA) or SPHINCS+ (SLH-DSA) this probability is negligible in the security parameter [15,16,5]. This channel subsumes all of Propositions 1–8 where the attack requires re-signing.

Receipt forge. The adversary constructs a receipt that satisfies $\text{VerifyReceipt}(R) = 1$ for a transition that did not occur. This requires either a BLAKE3 preimage or

second-preimage (2^{128} work minimum) or a collision in the OCEL 2.0 canonicalisation hash. The refusal states `ObservedOCELMissing`, `PathHashOnlyReceipt`, `FixtureMutationDetected`, and `SelfCertifiedAlignment` are engineering instantiations of this channel.

Boundary bypass. The adversary exploits an incomplete or incorrect boundary predicate B — a transition that should be refused is not in the boundary map. This is a *modelling* failure rather than a cryptographic one; its probability is bounded by the completeness of the process model. This is the primary risk for novel event types not yet in the declared model.

Policy gap. The adversary submits a change under a policy version p that does not encode the intended constraint. Version binding (p bound into σ) prevents policy downgrade, but it does not prevent *incorrect* policy content. Policy authoring is the human-in-the-loop component.

Validator compromise. The validator that computes $B(s, e)$ and issues the admission signature is itself compromised. Mitigations include HSM-anchored validator keys, validator version binding in σ , and out-of-band re-verification.

Object-model gap. The OCEL 2.0 object-type schema does not distinguish objects that are semantically different. For example, if all invoices share a type but the model does not enforce that an approval event binds the *same* invoice as the preceding receipt, object substitution (Proposition 3) may succeed. This is bounded by the precision of the object model.

Implementation bypass. Bugs in the admission gate allow $\text{Accept}(x) = 1$ to be returned incorrectly. This is a software quality risk, bounded by testing coverage, formal verification of the gate, and the boundary evidence requirements (refusal states `BoundaryEvidenceMissing`, `RuntimeObserverMissing`).

What This Decomposition Provides

The decomposition does not make the system invincible. It makes the system *falsifiable*: each channel has a specific type (cryptographic, algorithmic, modelling, operational, implementation), a specific bounded probability, and a specific mitigation path. The security claim is not “we cannot be hacked” but:

Unauthorized change cannot be accepted as valid enterprise state unless the adversary breaks one of the seven named assumptions. Everything outside the admitted universe is refused or residualized.

This is, deliberately, the language of formal security arguments rather than marketing.

6 Reference Implementation: wasm4pm

We instantiate the formal framework in *wasm4pm*, an open-source process mining platform [9] with 60 deterministic algorithms compiled from Rust to WebAssembly.

6.1 Architecture

wasm4pm has two layers with complementary roles:

- (1) A **Rust/WASM core** providing 60 process mining algorithms — discovery (DFG, Heuristic Miner, Inductive Miner), conformance checking (token replay, alignments), object-centric querying (OCPQ), and machine learning primitives. Algorithms are deterministic by construction: all RNG is seeded, all hash-map iteration is sorted. Same input implies bit-exact output, verified as a CI merge gate.
- (2) A **TypeScript monorepo** of eleven packages wrapping the WASM core with a CLI (**wpm**), Zod-validated configuration, an execution planner, OpenTelemetry spans, and a Supabase persistence layer for receipts and TrueX envelopes.

6.2 TrueX Envelopes as Concrete Admitted Changes

The *TrueX envelope* is the concrete realisation of a member $x \in A_{\text{enterprise}}$ in wasm4pm. Its fields map directly to the components of Definition 1:

Envelope field	Formal component	Type
<code>session_id</code>	a (actor/session)	<i>UUID</i>
<code>ocel2_batch_hash</code>	$h(O)$ (object log hash)	<i>BLAKE3 hex</i>
<code>receipt_hash</code>	$h(R)$ (receipt hash)	<i>BLAKE3 hex</i>
<code>admission_status</code>	$B(s, e)$ outcome	<i>ReceiptAdmitted</i>
<code>equivalence_class</code>	s (process state class)	<i>string</i>
<code>expected_path_hash</code>	$h(\text{model path})$	<i>BLAKE3 hex</i>
<code>truex_profile</code>	v (validator version)	<i>string</i>
<code>trace_id</code>	OTel span correlation	<i>string</i>

The `admission_status` field carries the string `ReceiptAdmitted` only when the verifier has checked all conjuncts of Accept. Self-certified alignment (`SelfCertifiedAlignment` refusal state) is architecturally prevented: the field is computed by the verifier and sealed into the envelope hash, never written by the producer.

6.3 Boundary Predicate as 13-State Refusal Taxonomy

The boundary predicate $B(s, e)$ is implemented as a taxonomy of thirteen non-overlapping refusal states, each a DENY-level gate:

- **Ingress validation:** `ObservedOCELMissing`, `ExpectedOCELMissing`, `PathHashOnlyReceipt`
- **Adversarial detection:** `ObservedTraceMutationWithoutBoundary`, `FixtureMutationDetected`
- **Envelope verification:** `RuntimeObserverMissing`
- **Cryptographic:** `ChallengeNonceMissing`, `ChallengeNonceMismatch`, `ObservedTraceNotChallengeBou`
- **Evidence:** `BoundaryEvidenceMissing`, `SummaryOnlyReceipt`
- **Authority:** `SelfCertifiedAlignment`
- **Hierarchy:** `ProofClassOverclaimed`

Each state corresponds to a distinct failure mode in Theorem 2 (receipt forge or boundary bypass channels).

6.4 Receipt Chain

Every `wpm run` writes a BLAKE3 receipt to `.wasm4pm/receipts/<run-id>.json`. The receipt carries: `run_id`, `command`, `input_hash`, `output_hash`, `status`, `payload` (full OCEL 2.0 log reference), and `git_commit`. Receipts are chained by including the prior receipt’s hash and can be synchronised to a central Supabase store via the `@wasm4pm/supabase` package, which implements the offline queue with a 5-attempt dead-letter cap.

6.5 Determinism as a Receipt Precondition

The WASM determinism guarantee — same input, bit-exact output across all platforms — is a necessary condition for receipt reproducibility. A non-deterministic analysis would produce different `output_hash` values on different machines, making receipts unverifiable. `wasm4pm` enforces determinism at the CI merge gate; stochastic algorithms use a fixed seed (42), configurable seed support is future work.

6.6 Operational Verification: Doctor Status

The `wpm doctor -supabase` command instantiates the Accept predicate verification as an operational status progression:

```

prepublish_only Credentials absent, Supabase unreachable, migrations not
                    applied, or service role key missing.
configured All credentials present, reachable, migrations applied.
live_verified Full smoke test passed: probe receipt upserted, read back, dead-
                    letter written, TrueX envelope ingested via Edge Function. A BLAKE3-
                    hashed runtime receipt is emitted to .wasm4pm/receipts/supabase_runtime.receipt.json.
```

The `live_verified` status is itself a receipt — a proof that the admission pipeline was operational at a given point in time.

7 Related Work

Process-aware information systems (PAIS). Van der Aalst et al. [1] laid the foundation for PAIS by defining workflow nets and soundness. Process mining as an analytic discipline was formalised in [2]. Our contribution moves process semantics from analytic post-processing to pre-admission control — a temporal inversion not present in prior PAIS or process mining work.

Conformance checking. Token replay [18] and alignment-based conformance [4] measure how well a recorded log fits a model. These approaches require a completed log. Our boundary predicate $B(s, e)$ enforces conformance *before* the event enters the log: it is conformance as a gate, not conformance as a measurement.

Object-centric process mining. Object-centric Petri nets [3] and the formal OCEL multigraph [6] provide the object-event semantics we exploit in Proposition 3. OCEL 2.0 is the standard log format [7]; OCPQ [13] provides object-centric querying that composes with our framework.

Blockchain-based audit trails. Several proposals use distributed ledgers to provide tamper-evident logs [14]. Our approach differs in two ways: (i) we do not require a distributed ledger — local BLAKE3 receipt chains plus a central store suffice — and (ii) we enforce admissibility *before* the change occurs, whereas blockchain audit trails record after the fact.

Attribute-based and policy-based access control. XACML [17] and attribute-based access control (ABAC) govern who may *read or initiate* a change. Our framework governs *whether the change is admitted as valid state*, which is orthogonal: an authorised actor may still produce a change that fails `VerifyReceipt` or $B(s, e) = 0$, and is therefore refused. Access control and change admissibility compose; neither subsumes the other.

Anomaly detection and fraud analytics. Probabilistic anomaly detection [12] and process deviation monitoring [8] identify suspicious behaviour statistically. Our approach is categorical, not probabilistic: either $\text{Accept}(x) = 1$ or not. The approaches are complementary: anomaly detection over the admitted receipt stream can surface policy gaps (the boundary bypass channel) that the framework does not close on its own.

Formal methods and model checking. Boundary predicates are closely related to guard conditions on Petri net transitions. Model checking tools (SPIN, nuXmv) could in principle verify the completeness of a boundary predicate specification — that is, that all illegal transitions satisfy $B(s, e) = 0$. We leave formal verification of the boundary predicate for future work; the present contribution is the architecture that makes such verification meaningful.

Post-quantum cryptography. The case for migrating enterprise systems to post-quantum signatures is surveyed in [5]. NIST’s standards [15,16] provide the concrete algorithms. To our knowledge, ours is the first work to apply post-quantum signing specifically to enterprise change admission receipts, motivated by the long verification horizon of compliance audit trails.

8 Conclusion

We have introduced the *Enterprise Change Admissibility Framework*, which extends van der Aalst’s admitted symbolic event semantics and boundary predicates into a governance law for all enterprise state change. The central claim — $\text{Accept}(x) \Rightarrow x \in A_{\text{enterprise}}$ — is not a claim of invincibility. It is a claim that the path from invalid input to accepted state is blocked by a conjunction of independently auditable predicates, and that every way that conjunction can be broken is a named, bounded assumption.

The framework’s primary contribution is a *temporal inversion*: process mining moves from after-the-fact analytics into pre-admission control. An enterprise that adopts this architecture does not merely log what its processes did; it requires that changes demonstrate process membership before they count.

Eight classes of adversarial mutation — raw writes, state skips, object substitution, backdating, replay, schema-valid fraud, rogue admin edits, and AI-agent hallucinated actions — are inaccessible to acceptance without breaking one of seven named assumptions. The wasm4pm reference implementation instantiates the framework with BLAKE3 receipt chains, OCEL 2.0 logs, thirteen refusal states, and post-quantum ML-DSA signing.

Limitations and future work.

- (1) **Boundary predicate completeness.** The 13 refusal states are motivated by observed attack patterns; a formal completeness proof against a threat model would close the boundary bypass channel. We leave this for future work using Petri net reachability analysis over the declared process model.
- (2) **Stochastic algorithm seeds.** wasm4pm’s stochastic algorithms use a fixed seed (42). Configurable, audited seeding would allow receipt-verified stochastic analysis.
- (3) **CLI–WASM trace correlation.** The TypeScript CLI and Rust/WASM core currently emit independent OTEL trace IDs. Distributed trace correlation across the boundary would close the implementation bypass channel for cross-boundary spans.
- (4) **Auditor endorsement.** The evidentiary weight of the receipt chain in a legal or regulatory context requires a Big Four or regulatory body to accept it as a compliance artefact. The framework is ready; the endorsement is institutional rather than technical.

The category shift. Every durable enterprise security problem is ultimately a change problem: *who changed the role, the approval, the contract, the model, the policy?* Current systems answer by auditing after the fact. This paper argues that the answer should be *by construction*: only admitted, signed, process-valid, receipt-bearing change can become enterprise state. That is Proof-Bound Enterprise Change, and it is a different primitive from validation, access control, or anomaly detection.

References

1. van der Aalst, W.M.P.: Verification of workflow nets. In: Application and Theory of Petri Nets 1997 (ICATPN 1997). Lecture Notes in Computer Science, vol. 1248, pp. 407–426. Springer (1997). https://doi.org/10.1007/3-540-63139-9_48, peer-reviewed. Springer link.springer.com/chapter/10.1007/3-540-63139-9_48 confirmed. Original WF-net soundness paper. Definitions: single source/sink, option to complete, dead-transition freedom, boundedness.
2. van der Aalst, W.M.P.: Process Mining: Data Science in Action. Springer, Berlin, Heidelberg, 2nd edn. (2016). <https://doi.org/10.1007/978-3-662-49851-4>, canonical textbook reference for DFG (Ch. 3), WF-net soundness, generalization metrics (Sec. 9.3), process trees (Ch. 6), and quality dimensions. Cited broadly across all algorithm families.
3. van der Aalst, W.M.P., Berti, A.: Discovering object-centric petri nets. *Fundamenta Informaticae* **175**(1–4), 1–40 (2020). <https://doi.org/10.3233/FI-2020-1946>, peer-reviewed journal. DBLP confirmed at dblp.org/db/journals/fuin/fuin175.html. IOS Press ebooks confirmed. arXiv:2010.02047. First formally complete process model for object-centric event data.
4. Adriansyah, A., van Dongen, B.F., van der Aalst, W.M.P.: Conformance checking using cost-based fitness analysis. In: Proceedings of the 15th IEEE International Enterprise Distributed Object Computing Conference (EDOC 2011). pp. 55–64. IEEE (2011). <https://doi.org/10.1109/EDOC.2011.12>, peer-reviewed. IEEE Xplore confirmed at ieexplore.ieee.org/document/6037560/. DBLP confirmed at dblp.org/pid/90/7976.html. First peer-reviewed paper introducing cost-based alignment for conformance; A* over product automaton of log trace and Petri net.
5. Bernstein, D.J., Lange, T.: Post-quantum cryptography. *Nature* **549**(7671), 188–194 (2017). <https://doi.org/10.1038/nature23461>, survey of post-quantum cryptography landscape. Motivates migration from ECDSA/RSA for long-lived signatures.
6. Berti, A., van der Aalst, W.M.P.: OC-PM: Analyzing object-centric event logs and process models. *International Journal on Software Tools for Technology Transfer* **25**, 1–17 (2023). <https://doi.org/10.1007/s10009-022-00668-w>, peer-reviewed journal. Springer confirmed at link.springer.com/article/10.1007/s10009-022-00668-w. arXiv:2209.09725. Formal multigraph definition of OC-DFG.
7. Berti, A., Koren, I., Adams, J.N., Park, G., et al.: OCEL (Object-Centric Event Log) 2.0 specification. arXiv:2403.01975v1 [cs.DB] (2024), <https://arxiv.org/abs/2403.01975>, oCEL 2.0 specification. 13 named authors. Adds O2O relationships, dynamic object attributes, qualified arc types; three exchange formats: SQLite, XML, JSON. arXiv preprint of spec document.
8. Bose, R.P.J.C., van der Aalst, W.M.P., Žliobaitė, I., Pechenizkiy, M.: Dealing with concept drifts in process mining. *IEEE Transactions on Neural Networks and*

- Learning Systems **25**(1), 154–171 (2014). <https://doi.org/10.1109/TNNLS.2013.2278313>, peer-reviewed. IEEE TNNLS journal. Canonical PM concept drift paper; grounds detect_drift and compute_ewma.
9. Chatman, S.: wasm4pm: A deterministic WebAssembly process mining platform. GitHub repository and npm package (2026), <https://github.com/seanchatmangpt/wasm4pm>, 60 deterministic process mining algorithms compiled from Rust to WebAssembly. Implements the Enterprise Change Admissibility Framework via TrueX envelopes, BLAKE3 receipt chains, and 13-state boundary predicate taxonomy.
 10. Chesterton, G.K.: Orthodoxy. Dodd, Mead and Company, New York (1908), original statement of the Chesterton’s Fence principle (Chapter 4): do not remove a fence until you understand why it was built.
 11. Ghahfarokhi, A.F., Park, G., Berti, A., van der Aalst, W.M.P.: OCEL: A standard for object-centric event logs. In: New Trends in Database and Information Systems — ADBIS 2021 Short Papers, Doctoral Consortium and Workshops: DOING, SIMPDA, MADEISD, MegaData, CAoNS. Communications in Computer and Information Science, vol. 1450, pp. 169–175. Springer (2021). https://doi.org/10.1007/978-3-030-85082-1_16, peer-reviewed. Springer confirmed at link.springer.com/chapter/10.1007/978-3-030-85082-1_16. SIMPDA workshop at ADBIS 2021, Tartu, Estonia, August 24–26, 2021. OCEL 1.0 format specification. Authors: 4 (not 3 as in earlier stub).
 12. Ko, J., Comuzzi, M.: Detecting anomalies in business process event logs using statistical leverage. Information Sciences **549**, 53–67 (2021), peer-reviewed journal. Information-theoretic edge-frequency anomaly scoring; grounds ml_anomaly algorithm.
 13. Küsters, A., van der Aalst, W.M.P.: OCPQ: Object-centric process querying and constraints. In: RCIS (1) 2025. pp. 383–400 (2025). https://doi.org/10.1007/978-3-031-92474-3_23, peer-reviewed conference (RCIS 2025, 19th International Conference). arXiv:2506.11541. Formal Definitions 1–9 for OCPQ query tree semantics. Rust implementation at github.com/aarkue/ocpq.
 14. Lopez-Pintado, O., García-Bañuelos, L., Dumas, M., Weber, I., Ponomarev, A.: Caterpillar: A business process execution engine on the Ethereum blockchain. In: Information Systems. vol. 90, p. 101470. Elsevier (2019). <https://doi.org/10.1016/j.is.2019.07.009>, blockchain-based process execution. Provides immutable audit trail. Contrasts with our approach: we do not require a distributed ledger.
 15. National Institute of Standards and Technology: Module-lattice-based digital signature standard (ML-DSA). Tech. Rep. FIPS 204, U.S. Department of Commerce (2024), <https://doi.org/10.6028/NIST.FIPS.204>, based on CRYSTALS-Dilithium. Specifies ML-DSA-44, ML-DSA-65, ML-DSA-87. Primary signing algorithm recommended for enterprise receipt chains.
 16. National Institute of Standards and Technology: Stateless hash-based digital signature standard (SLH-DSA). Tech. Rep. FIPS 205, U.S. Department of Commerce (2024), <https://doi.org/10.6028/NIST.FIPS.205>, based on SPHINCS+. Hash-based; conservative fallback when lattice assumptions are uncertain.
 17. OASIS: eXtensible access control markup language (XACML) version 3.0. Tech. rep., OASIS Standard (2013), <https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html>, attribute-based access control standard. Governs who may initiate change; orthogonal to admissibility of the change itself.
 18. Rozinat, A., van der Aalst, W.M.P.: Conformance checking of processes based on monitoring real behavior. Information Systems **33**(1), 64–95 (2008). <https://doi.org/10.1016/j.is.2007.12.001>.

[org/10.1016/j.is.2007.07.001](https://doi.org/10.1016/j.is.2007.07.001), peer-reviewed journal. ScienceDirect confirmed at [sciencedirect.com/science/article/abs/pii/S030643790700049X](https://www.sciencedirect.com/science/article/abs/pii/S030643790700049X). DBLP confirmed at dblp.org/pid/20/5156.html. Canonical token replay paper; fitness formula $\text{fitness} = 1 - (\text{missing} + \text{consumed}) / (\text{produced} + \text{remaining})$ implemented verbatim in wasm4pm.